Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3755881.3755891

RESEARCH-ARTICLE

To Split or to Merge? Exploring Multi-modal Data Flexibly for Failure Classification in Microservices

XIUHONG TAN, Changsha University of Science and Technology, Changsha, Hunan, China

YUAN YUAN, National University of Defense Technology China, Changsha, Hunan, China

TONGQING ZHOU, National University of Defense Technology China, Changsha, Hunan, China

SHIMING HE, Changsha University of Science and Technology, Changsha, Hunan, China

YUQI LI

JIAN ZHANG

Open Access Support provided by:

National University of Defense Technology China

Changsha University of Science and Technology

PDF Download
3755881.3755891.pdf
18 March 2026
Total Citations: 0
Total Downloads: 235

Published: 27 October 2025

Citation in BibTeX format

Internetware 2025: the 16th International Conference on Internetware
June 20 - 22, 2025
Trondheim, Norway

Conference Sponsors:
SIGSOFT

https://doi.org/10.1145/3755881.3755891
ISBN: 9798400719264

To Split or to Merge? Exploring Multi-modal Data Flexibly for Failure Classification in Microservices

Xiuhong Tan*
Changsha University of Science and Technology
Changsha, China
tanxh@stu.csust.edu.cn

Yuan Yuan*
National University of Defense Technology
Changsha, China
yuanyuan@nudt.edu.cn

Tongqing Zhou†
National University of Defense Technology
Changsha, China
zhoutongqing@nudt.edu.cn

Shiming He
Changsha University of Science and Technology
Changsha, China
smhe_cs@csust.edu.cn

Yuqi Li
National Supercomputer Center in Tianjin
Tianjin, China
liyq@nscc-tj.cn

Jian Zhang
National Supercomputer Center in Tianjin
Tianjin, China
zhangjian@nscc-tj.cn

Abstract

Classifying failures for automated diagnosis is critical for maintaining the reliability of ever-increasing microservice systems. Existing failure classification methods focus solely on analyzing one specific data modality (e.g., metrics) or feeding multi-modal data (e.g., logs and traces) in a one-fit-all model. This would lead to biased inference as different modalities have differentiated proportions and manifest distinct sensitivities and patterns in failures of varied runtime contexts. This work proposes a novel failure classification framework for microservices, named AgileMFC, with the basic idea of flexibly learning and assembling the expertise of multi-modal monitoring data. The flexibility is attained with the split-and-merge design of data representation, where it assigns each modality a different gate that routes it to dedicated expert networks and uses a sharing gate to facilitate controllable knowledge interaction among modalities. Meanwhile, AgileMFC trains the representation networks and a classifier in consecutive stages with concatenated and weighted feature fusion, thus decoupling task-agnostic feature extraction with task-specific classification. Experiments on three datasets show that AgileMFC outperforms four baselines w.r.t., precision, recall, and F1-score on failure classification.

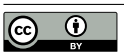
CCS Concepts

• **Computer systems organization** → **Reliability**; • **Software and its engineering** → *Maintaining software*; • **Computing methodologies** → **Machine learning**.

Keywords

Microservice systems, failure classification, multi-modal data analysis

*Both authors contribute equally to this research.
†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
Internetware 2025, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1926-4/25/06
https://doi.org/10.1145/3755881.3755891

ACM Reference Format:
Xiuhong Tan, Yuan Yuan, Tongqing Zhou, Shiming He, Yuqi Li, and Jian Zhang. 2025. To Split or to Merge? Exploring Multi-modal Data Flexibly for Failure Classification in Microservices. In *the 16th International Conference on Internetware (Internetware 2025), June 20–22, 2025, Trondheim, Norway.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3755881.3755891

1 Introduction

With the popularity of cloud-native applications, microservice architecture has been widely adopted to decouple ever larger systems into independent units for agile development [5, 19]. A microservice system is typically presented as a large-scale distributed architecture comprising loosely coupled instances (e.g., physical machines or containers). However, as the number of instances increases to cater to growing user demand, the invocation relationship among instances becomes increasingly complex. This would aggravate the propagation of failures and decrease the stability of the whole system, causing terrible user experiences or even economic damage [13]. Hence, effectively diagnosing failures is critical for troubleshooting and maintaining a short Mean Time to Repair (MTTR).

In the failure diagnosis spectrum, failure classification (a.k.a, failure triage) is used to identify the type of potential problems for dedicated downstream mitigation [17], thus being considered an essential step. For example, by identifying a failure type of intensive workload (e.g., I/O saturation), the operator could complement a load balancer for corresponding instances. Technically, failures can be classified by parsing and analyzing the monitoring data of microservice systems, which mainly includes the chronological records of system events and transactions (i.e., logs), quantitative indicators of system performance (i.e., metrics), and the flow of requests through microservices (i.e., traces). These modalities of data are usually represented as events[1] in different structures and forms, e.g., texts for logs, time series for metrics, and tuples for traces, as shown in Table. 1.

Existing failure classification methods rely on either isolated single-modal analysis (i.e., *split*) or one-fit-all multi-modal analysis (i.e., *merge*), falling into two extremes. On the one hand, single-modal classification [1, 11, 23], based on parsing texts, anomaly

[1]We use the term event for a log/metric/trace record during failure.

detection, and correlation analysis of logs, metrics, and traces respectively, is an intuitive choice as different modalities are collected independently. However, relying on a single type of data may fail to capture the full characteristics of failures, leading to incomplete analysis. Furthermore, heterogeneous microservice systems mean that failures can manifest in a variety of ways, so single-modal data may not adequately reflect this diversity. On the other hand, there have been attempts to integrate different modalities of monitoring data in failure classification recently [13, 24, 26]. Although existing multi-modal classification methods overcome single-modal's incompleteness drawback with complementary data, this work notes that they go to another extreme by fitting the representation of different modalities in one model.

For example, as the State-of-The-Art (SoTA) design along the multi-modal line, diagFusion [24] uses fasttext for embeddings of logs, metrics, and traces, and introduces Graph Neural Network (GNN) for feature representation towards classification. Representing each modality in the same form would **facilitate a biased model given that different modalities have distinct sensitivities and different number of events on a specific failure**. As in the example of Table. 1, empirically, an operator would identify the failure type "Access denied" by the log information at 17:38. Forcing the representation of log "access permission exception" to be aligned with metric "CPU" and trace "s1→ s2" would degrade its indicative capability. Meanwhile, diagFusion's use of GNN requires the presentation of events from all microservice instances when a failure occurs, which would fail in practice because no events would be triggered for those normal instances. Hence, this work is motivated to design **a flexible assembling of different modality knowledge for unbiased classification of microservice failure**.

Table 1: An example of the multi-modal events/data during a failure. $RT_{S_1 \to S_2}$ denotes the response time when S_1 calls S_2, "–" means no anomaly is found or data is lost in that modality, and "↑" and "↓" indicate the value is out of the standard range.

Time	Log	Metric	Trace
17:12	-	Mem↑	$RT_{S_1 \to S_2}$=1.8s
17:31	-	CPU↓	-
17:32	INFO | User with ID ** has been ...	-	$RT_{S_2 \to S_3}$=1.9s
17:38	ERROR | ... access permission exception ...	-	-
17:44	-	Net↓	$RT_{S_1 \to S_4}$=3.5s

For this goal, we have to handle two aspects of challenges: *1) How to take balance between split and merge?* In terms of microservice monitoring data representation for different modalities, split/independency maintains modality sensitivity but is incomplete in depicting the failure, while merge facilitates comprehensive understanding at the cost of weakening modalities' differential influences. A flexible design is expected to retain the advantages of both. *2) How to accommodate representation and classification?* Representation for feature extraction is task-agnostic while classification, as its subsequent step, is not. Learning them together in an end-to-end way may force representations for different modalities to share the same classification loss, gained from the training samples, during optimization, implicitly depriving their sensitivities.

This work proposes *AgileMFC*, a novel framework that leverages the strengths of each modality of monitoring data for microservice failure classification. Specifically, for challenge 1, AgileMFC uses a series of parallelized and structural-adaptive channels (deep networks) for representing log, metric, and trace events, and regards these channels as modality-sensitive experts. It then merges these experts together through a Multi-gate Mixture-of-Experts (MMoE) architecture, where each modality is mapped to one modality gate that tells it how much (i.e., weight/attention) it can trust in the representation of each network. Meanwhile, AgileMFC introduces a sharing gate for interacting general representation knowledge among experts. Such a **flexible split&merge design** guarantees that the learning of different experts is agnostic to the event components during microservice failures and avoids the overwhelming effects (in existing multi-modal methods) in the representations of some modalities to others. As to challenge 2, AgileMFC adopts a two-stage training strategy to decouple representation and classification learning. First, the expert-output features of multi-modal events are fused via concatenation and linearly mapped to the label vector size for calculating representation losses and tuning the representation networks. Second, a classifier network is connected to the learned experts and tuned with a classification loss, during which stage the expert networks are frozen.

The flexible design of AgileMFC endows it with **SoTA performance in failure classification**[2], which is validated on three datasets (one from a real-world microservice system and two from open-source platforms). The major contributions are as follows:

- The proposed AgileMFC framework adapts MMoE with both modality-aware and sharing gating mechanisms for unbiased events feature extraction in microservice failure, presenting a flexible trade-off between splitting and merging data modalities in representation.
- This work adopts a two-stage training design for the consecutive-yet-decoupled learning of the representation networks and the classifier in AgileMFC. Concatenated and weighted fusion strategies are adopted for different training stages, respectively.
- The effectiveness and performance of AgileMFC are evaluated by comparing with four failure classification baselines on three datasets. Experimental results show that AgileMFC achieves the SoTA performance on classification precision, recall, and F1-score with an average (w.r.t., three datasets) improvement of 12.4%, 9.1%, and 11.8% over the 2nd-best baseline.

2 Related Work

Failure type classification is a crucial task of failure diagnosis in large-scale microservice systems. Many research papers focus on this domain [1, 13, 16, 22–24], which can be categorized into single-modal-based and multi-modal-based methods.

[2]Code is available at https://github.com/aiopsUser1012/AgileMFC

2.1 Single-modal-based Failure Classification

Log-based failure classification: Semi-structured logs record event information during the runtime of a system. Existing methods parse and classify the potential failures in log records by vectorizing and templating logs into features. Among them, LogCluster [11] parses log information into log events, and groups them using the Agglomerative Hierarchical clustering technique. It then determines the failure type of an incoming event based on the cosine similarity between it and the clusters. Cloud19 [23] trains a word2vec model to vectorize logs and adopts a supervised classifier for failure type classification. Although the log entails semantic events in the system, the recording protocols cannot provide the invocation and system context, which may lead to missing or misclassification.

Metric-based failure classification: Metrics carry fine-grained information, especially resource usage, about the operational state of a system, so are widely used to monitor system health and performance. DBSherlock [22] combines domain knowledge and user feedback in analyzing historical metrics data and identifying probable failure types. By building a KNN model for the normal behavior of metrics, CloudPD [15] generates signatures for different online failure types based on expert knowledge and determines the failure type of incoming metrics by comparing them with the existing signatures. Arvalus [14] regards instance subseries as nodes and their dependencies as edges, uses graph convolutional neural networks (GCNNs) to learn node and edge features, and classifies failures based on the graph representation. However, metrics alone are susceptible to unexpected variations (unstable in nature) and lack context semantics, which would mislead failure triage.

Trace-based failure classification: The trace-based methods accomplish failure tracing and classification by analyzing the calling or deployment dependencies between microservice instances. MicroHECL [12] designs specialized detection models for different types of anomalies (performance, reliability, and traffic), each of which corresponds to a specific quality of service metric and a different direction of anomaly propagation. MicroHECL then categorizes failure types based on the identified anomalies. Traces entail the inter-microservice relationship, whose features are more effective in understanding coarse-level failures with explicit propagation chains than those rooted in one service. Hence, merely referring to the traces would lead to an incomplete understanding of failures.

2.2 Multi-modal-based Failure Classification

Single-modal-based methods may omit certain types of information and thus are more likely to increase the risk of failure. In contrast, combining multi-modal data offers a more complete view for checking the microservice system, allowing for more accurate failure classification [27]. CloudRCA [26] builds a Knowledge Information Hierarchical Bayesian Network (KHBN) to identify anomalies in both metrics and logs and correlate them with specific predefined failure types. Meanwhile, MicroCBR [13] uses metrics, logs, and traces to build a spatio-temporal failure knowledge graph and generates feature fingerprints for each historical failure. Subsequently, during the online phase, the features of new failures are compared with the fingerprints of known failures through a hierarchical inference mechanism to determine their categories. Zhang et al. [24] provides observations for the modest performance of identifying microservice failures based on single-modal data of the Generic AIOps Atlas dataset. Then they present DiagFusion, which encodes logs, traces, and metrics with fasttext, represents each instance with GNN, and feeds the representations to an MLP for failure classification. Although multi-modal data are involved, their embedding into vectors is handled by a universal model, which implicitly enforces the representation of multi-modal data to failures in the same feature space.

3 Preliminaries

In this section, we first introduce the background of the microservice system and some preliminaries on failure classification. Then we define the problem of failure classification with multi-modal inputs.

3.1 Monitoring the Microservice System

A microservice system consists of independently deployable services, each responsible for accomplishing a specific task. One microservice could be launched many times and each of these running entities is called an instance. These instances communicate with each other using lightweight protocols, typically HTTP or message queues [9]. Since each service can be updated or extended individually, the microservice architecture allows developers to develop, test, and deploy freely (a.k.a., DEVOps). In practice, microservice systems refer to three main types of data to monitor the health status:

Log: A log message usually consists of fields such as timestamp, verbosity level, and raw information [6]. Wherein, verbosity levels are usually categorized into INFO, WARN, DEBUG, and ERROR, indicating the severity. Log template parsing technology (e.g., Drain [6]) and visual management tools such as ELK (Elasticsearch, Logstash, and Kibana) can help operators to understand log messages more efficiently.

Metric: Metrics are real-number values that depict the status of microservice instances. Well-known metrics include CPU usage, memory consumption, request rates, error rates, etc. They are collected at predetermined intervals to form time series. When specific metrics deviate from the expected criteria, automated alerts would be triggered.

Trace: A trace tracks the flow of requests among instances of a microservice system. It can be formulated as a tree structure, with instances as nodes and invocations as edges. Given such a tree, it is easy to determine the location of the current request in the invocation hierarchy, as well as its upstream and downstream nodes. Typically, besides the callers and callees, a trace carries information about the invocation time, request status, latency, status codes, etc.

3.2 Failure Classification

Decoupling services into units aggravates failure propagation [2], in particular for ever-larger microservice systems, where small failures can have a cascading effect, making diagnosis and recovery more critical.

Failure classification denotes the process of categorizing failures according to the type, severity, or influence on system performance. It is essentially used as an upstream task for developing effective failure diagnosis and recovery strategies. When a system reports a

failure, the operator would first refer to the context (manifested in monitoring data) and then decide the general diagnostic direction for troubleshooting using the failure classification model. Hence, effective failure classification is critical for efficient diagnosis.

W.l.o.g., this work studies failure type classification. Large-scale microservice systems may suffer from many different types of failures, which can be roughly classified as hardware failures, software errors, and network problems [25]. Hardware failures often arise from resource scheduling problems, where incorrect resource allocation or heavy system loads can make a service run low on CPU, memory, or network resources, further causing service interruptions or crashes. Software errors generally include system bottlenecks, poorly query statements, or code bugs. Network problems involve issues such as latency and connection failures that can disrupt inter-service communication and cause cascading failures in highly interdependent microservices. Note that the failure types could be further refined according to the needs of implementation and the availabilities of labels, i.e., the type of hardware failure could be refined as misconfiguration, system component damage, intensive workload, resource exhaustion, and etc.

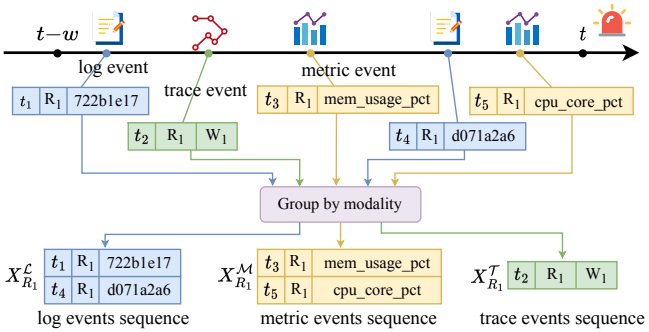


Figure 1: The process of event sequence generation. The icon of each modality represents the raw data, the icon at t indicates a failure and w is the length of the observation window.

3.3 Problem Formulation

Failures in microservice systems could cause a series of events given the cascading invocation characteristics. Meanwhile, the symptom events and modality may vary in contexts as microservices are developed and maintained independently with hardly consistent running patterns and recording protocol. To avoid biasedly relying on partial modalities and instances, this work attempts to use multi-modal monitoring data in the failure scene for supervised learning of failure classification.

Specifically, we consider a large-scale microservice system with Q instances, and logs, metrics, and traces are aggregated for each instance. For a failure F_t that occurs at time t and a w-length observation window ahead, we can attain its contextual multi-modal data, $\mathrm{F}_{[t-w,t]} = \{(X_i^{\mathcal{L}}, X_i^{\mathcal{M}}, X_i^{\mathcal{T}})\}_{i=1}^{Q}$, where $X_i^{\mathcal{L}}, X_i^{\mathcal{M}}, X_i^{\mathcal{T}}$ are the event sets of corresponding modalities (i.e., logs, metrics, traces) for the i-th instance. As shown in Fig. 1, the multi-modal event sequences for instance R_1 in the window before failure are grouped by modality into $X_{R_1}^{\mathcal{L}}, X_{R_1}^{\mathcal{M}}$, and $X_{R_1}^{\mathcal{T}}$. The targeted framework is thus to gain a parameterized model that predicts the discrete category of a failure:

$$\mathcal{F}(\theta) : \mathrm{F}_{[t-w,t]} \rightarrow y.$$

4 Design of AgileMFC

This section presents AgileMFC's design on event (monitoring data) pre-processing, feature extraction (representation), and classification. It also describes the two-stage training details.

4.1 Framework Overview

An overview of the AgileMFC framework is shown in Fig. 2. After capturing a failure, it identifies multi-modal events from raw monitoring data and submits them to pre-processing, representation, and classification in phases. During pre-processing, AgileMFC embeds events into a vector space, where positional encoding is added to each event's embedding to encode their sequential relationships. Then, the embedding vectors of events are fed into a modality-aware representation network, which uses multiple experts to extract features for each event and combines their opinions based on the weight estimation of both a sharing gate and a modality-specific gate. After this phase, each event will have one feature vector. Finally, the fusion-based classification network fuses the event features based on event weights w.r.t. records in historical failures and determines the failure type of the fused vector via a classifier.

4.2 Monitoring Data Pre-processing

4.2.1 Event Formalization. AgileMFC obtains the three types of monitoring data by identifying events from the raw data using anomaly detection and template parsing, and attaching them with timestamps.

Log Event Parsing. We apply the Drain [6] tool, which has demonstrated its efficacy in real-world scenarios, to extract log events in the time window of failure. Drain constructs a fixed-depth parse tree to separate static and dynamic components of log messages. For example, given the log message "TravelServiceImpl: [Travel Service][Get Route By Id] Route ID uff1a92708982-77af-4318-be25-57ccb0ff69ad", "TravelServiceImpl: [Travel Service][Get Route By Id] Route ID ******" is the static template, "uff1a92708982-77af-4318-be25-57ccb0ff69ad" is the dynamic portion. After isolating the static template from the dynamic variables, we generate a unique identifier (i.e., template id) for each log template by applying a hash function to the template string. The final log events are stored as tuples in the form of < *timestamp, instance-id, template-id* >, with each entry representing a specific instance event.

Metric Event Parsing. For monitoring the operating state of instances, metrics are meticulously tracked and recorded at the individual instance level. We use the 3-sigma technique to detect abnormal metrics in the time window. When a metric's value surpasses the upper threshold, it is classified as an "upward" anomaly, indicating a significant increase. Conversely, if the metric value drops below the lower bound, it is flagged as a "downward" anomaly, reflecting a notable decrease. The extracted metric events are stored in a tuple format < *timestamp, instance-id, metric-name, anomaly-type* >, capturing the specific event and its anomaly type for analysis.

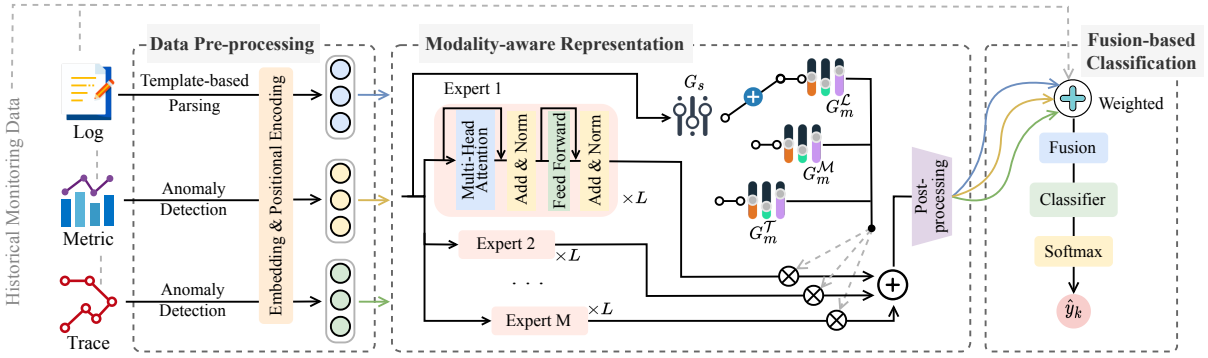


Figure 2: The framework of AgileMFC. Wherein, the representation module extracts a feature vector for each event in turn with the switch connecting to the modality gate corresponding to the event modality.

Trace Event Parsing. Traces are important for reconstructing the causality and flow of operations within a microservice system. Instances will be called when the service processes a request from the user and the timing gap between an instance being invoked and getting a response is called response time. Meanwhile, the system will return a status code to record the result of a request. We form the response time and the number of each status code as two time series, and then apply 3-sigma for anomaly detection on them. That is, we determine such a set of caller and callee instances as anomalous when the response time or the number of status codes exceeds three standard deviations from their mean values. The extracted trace events are recorded in the form of tuples < *timestamp, caller-instance-id, callee-instance-id* >, providing a structured representation of the event flow between instances.

The above extraction provides events from different modalities. We group events by instance-id, then reorder the events from different modalities in the same instance by timestamp, and finally obtain the event sequence. Fig. 1 shows the processing from events to event sequences. The j-th multi-modal event is represented $d_j^{\mathcal{L}}$, $d_j^{\mathcal{M}}$, and $d_j^{\mathcal{T}}$. The event sequence consisting of the same modality (e.g., log) is denoted by $X^{\mathcal{L}} = \{d_1^{\mathcal{L}}, ..., d_j^{\mathcal{L}}\}$. Subsequently, we can obtain the event sequence of the instance i, which $X_i = [X_i^{\mathcal{L}}, X_i^{\mathcal{M}}, X_i^{\mathcal{T}}]$.

4.2.2 Event Embedding. The raw event texts, although unified into a similar tuple format, are still unstructured and sparse in representations. Therefore, we need to transcode the event texts for further representation learning. AgileMFC converts these texts into continuous and structured representations through event semantic embedding and positional coding.

Semantic Embedding. We use a neural network layer to convert the one-hot vector of each event text into a dense vector. This process maps discrete events into a continuous vector space where semantically similar events are close to each other. For example, for the event sequence of log $X^{\mathcal{L}}$, we apply the embedding function to obtain the corresponding embedding vector $E^{\mathcal{L}} = \{e_1^{\mathcal{L}}, ..., e_j^{\mathcal{L}}\}$. The embedding layer is initialized using random weights and fine-tuned during training.

Positional Encoding. To enable the model to distinguish between events based on their positions in the sequence, we apply positional encoding to the embedding vector for each event. Specifically, for an event at position pos in the sequence, we generate a positional encoding vector $PE(pos)$ using sinusoidal functions:

$$PE(pos, 2i) = \sin(pos/10000^{2i/dim})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/dim}), \quad (1)$$

where i is the dimension index and dim is the embedding dimension. Thus, we can obtain a combined representation that preserves both semantic and positional information.

4.3 Modality-aware Representation

With monitoring data pre-processing, AgileMFC has mapped events with different modalities into a uniform form of embedding. However, these embeddings are inherently heterogeneous due to the diversity of modalities, making it challenging to collectively correlate events for diagnosing the failure. In this part, we propose to use deep features to elaborate on homogeneous representations for different modalities.

For this, AgileMFC uses events with failure types labels in historical failures for supervised learning of a modality-aware representation network. Considering that different modalities have different sensitivities for specific failures, AgileMFC involves multiple expert networks, each of which is believed to have differential expertise in diagnosing different modalities. Alongside this split opinion expressing step (§ 4.3.1), it further determines the reliance and confidence of different experts' suggestions and merges them with deliberate gating networks (§ 4.3.2).

4.3.1 Expert Networks. Inspired by the recent advances of MoE architecture and the multi-channel input characteristics, AgileMFC uses m expert networks (the number of experts is tunable) in representation. For network structure, recurrent neural networks (RNNs) have been widely used to capture the temporal pattern in previous work, [3, 10], while convolutional neural networks (CNNs) are commonly used to extract local traits [20, 21]. However, RNNs suffer from computational inefficiency due to their sequential processing nature, while CNNs are limited in modeling long-range dependencies across multiple.

In this design, AgileMFC employs Transformer blocks [18] as the realization of an expert. The Transformer architecture, unlike sequential processing of RNNs, computes all positions simultaneously, which could accelerate training and inference. The self-attention mechanism allows each expert to focus on the failure-sensitive parts of an event vector. Meanwhile, the using of multiple experts implicitly reduce the risk that one (or more) expert(s) cannot properly understand an event (i.e., without expertise) like the others.

Specifically, each expert f consists of L stacked Transformer encoder layers (the number is tunable). Given an input log event embedding $e_j^{\mathcal{L}}$, the output of the m-th expert is computed as:

$$f_m(e_j^{\mathcal{L}}) = \text{TransformerBlock}^L(e_j^{\mathcal{L}}), \tag{2}$$

where each Transformer block comprises multi-head self-attention and feed-forward sublayers with layer normalization and residual connections [18]. The outputs of all M experts are in the form of:

$$\text{expertOut}(e_j^{\mathcal{L}}) = [f_1(e_j^{\mathcal{L}}), f_2(e_j^{\mathcal{L}}), ..., f_M(e_j^{\mathcal{L}})]. \tag{3}$$

Note that experts perform encoding in an event-wise manner (e.g., Equation 3 outputs the i-th log event in the sequence), which handles the input sequence of events in turn. Such a split design allows different experts to gain differential representation expertise on different modalities, e.g., temporal dependence in logs and spatial correlation in metrics.

4.3.2 Modality & Sharing Gate. After the split encoding of experts, AgileMFC uses gate networks to integrate expert outputs for each event. The original MMoE network employs a single gate per task to blend expert outputs, which struggles to balance single-modal and cross-modal features in the multi-modal context. Given this, we design a quadruple gating mechanism that maintains an individual gate for each modality (i.e., $G_{\text{m}}^{\mathcal{L}}$ for log, $G_{\text{m}}^{\mathcal{M}}$ for metric, and $G_{\text{m}}^{\mathcal{T}}$ for trace), and shares one global cross-modal gate (G_{s}). A switch is used before the gating networks, which connects the sharing gate with the modality gate that corresponds to the input event's modality, i.e., only one modality gate is activated for merging an event's expert outputs. The gating process of these two types of gates is calculated as follows:

$$G_{\text{s}}(e_j^{\mathcal{L}}) = W_{\text{s}} \cdot e_j^{\mathcal{L}},\ G_{\text{m}}^{\mathcal{L}}(e_j^{\mathcal{L}}) = W_{\text{m}}^{\mathcal{L}} \cdot e_j^{\mathcal{L}}, \tag{4}$$

where $W_{\text{s}}, W_{\text{m}}^{\mathcal{L}} \in \mathbb{R}^{dim \times M}$, both W_{s} and W_{m} are trainable matrices. Similarly, it uses matrices $W_{\text{m}}^{\mathcal{M}}$ and $W_{\text{m}}^{\mathcal{T}}$ for gating metric and trace events, respectively. The final gating weights for the log modality are computed as:

$$\text{gateOut}(e_j^{\mathcal{L}}) = \text{softmax}(\alpha \cdot G_{\text{s}} + (1-\alpha) \cdot G_{\text{m}}^{\mathcal{L}}), \tag{5}$$

where α is the weight coefficient of the sharing gate and the j-th element in $\text{gateOut}(e_j^{\mathcal{L}})$ counts for the weight/reliance of the output of the j-th expert.

4.3.3 Post-processing. Post-processing merges the expert features via weighted sum and transforms them through linear processing. Specifically, we use a simple MLP structure that consists of two fully connected layers with ReLU activation and dropout regularization in between. The post-processing modality features $z_j^{\mathcal{L}}$ of j-th event is computed as follows:

$$z_j^{\mathcal{L}} = \text{M}_{out}(\text{expertOut}(e_j^{\mathcal{L}}) \odot \text{gateOut}(e_j^{\mathcal{L}})), \tag{6}$$

where $\odot$ denotes the Hadamard product (i.e., element-wise product).

4.4 Fusion for Classification

Based on the event representation learned in § 4.3, the classification network subsequently fuses the representations in the sequence to attain an overall failure representation for type classification.

In particular, this phase first calculates weight scores for events in the sequence by using TF-IDF on historical failure events and current events. Given a collection of events about P historical failures $T = \{t_1, t_2, ..., t_p\}$, each t_i represents an event sequence, we first construct a term-frequency matrix $X \in \mathbb{R}^{P \times V}$:

$$X_{i,j} = \text{count}(t_i, v_j), \tag{7}$$

where V is the size of the vocabulary, v_j is the j-th vocabulary term and $\text{count}(t_i, v_j)$ counts its occurrences in t_i. We then apply TF-IDF weighting to obtain the importance matrix $W = \text{TF-IDF}(X)$. This weighting scheme emphasizes events that are frequent in this failure and rare across the entire dataset, effectively capturing the specific characteristics of the failure.

A failure involves multiple instances, each containing several events from different modalities. Hence, we compute the failure representation H by aggregating all event representations in all instances weighted by their TF-IDF scores:

$$H = \sum_{i=1}^{N} \sum_{k=1}^{K} w_{(i,k)} \cdot z_{(i,k)}, \tag{8}$$

where N is the number of instances, K is the number of events, $w_{(i,k)}$ (an element in W) is the TF-IDF weight for event k of instance i, and $z_{(i,k)}$ is the representation for event k of instance i, gained from Equ. 6.

This polymerization ensures that H captures the full context of a failure, assembling information about all relevant events from different instances and modalities. Finally, the predicted failure type is obtained through a three-layer neural linear classifier: $\hat{y} = \text{LinearClassifier}(H)$.

4.5 Two-stage Training for Decoupled Learning

Considering that representation is agnostic to task while classification is, we propose to train these two modules independently by first training the representation with shortcut loss and then freezing it to train the classifier secondly.

4.5.1 Tuning the Representation Network. We directly connect the output of the representation network to a shortcut output for loss evaluation and optimization. A concatenation is applied to the three modality features (i.e., $z^{\mathcal{L}}$, $z^{\mathcal{M}}$ and $z^{\mathcal{T}}$) to obtain a flat feature. Then, a linear transformation is used to map them into a compact failure classification prediction.

The transformation process can be described as follows:

$$Z = W_f \cdot [z^{\mathcal{L}} \| z^{\mathcal{M}} \| z^{\mathcal{T}}] + b_f, \tag{9}$$

where $[\cdot\|\cdot]$ represents the concatenation operation, W_f and b_f are the mapping weight matrix and bias, respectively.

The combined representation Z encapsulates the joint understanding of all events in a failure. Next, we independently optimize the modality-aware representation network to ensure it captures

discriminative cross-modal patterns. This tuning process is guided by an auxiliary representation loss:

$$\mathcal{L}_{rep} = -\frac{1}{P}\sum_{i=1}^{P}\sum_{c=1}^{C} y_{i,c} \cdot \log\left(Z_{i,c}\right), \tag{10}$$

where P is the number of historical failures, C is the number of failure types, and y is the failure type. This independently optimized procedure allows for distinguishable features for different failure types and unbiased training of experts and modality gates.

4.5.2 Tuning the Classifier. In this stage, with the representation network frozen, the classifier network is optimized by minimizing the following cross-entropy loss:

$$\mathcal{L}_{clf} = -\frac{1}{P}\sum_{i=1}^{P}\sum_{c=1}^{C} y_{i,c} \cdot \log\left(\hat{y}_{i,c}\right), \tag{11}$$

In addition, AgileMFC incorporates grid search during training to determine the optimized hyperparameters for the number of experts and the number of blocks. By exhaustively searching and evaluating all possible combinations of hyperparameters, the model is adaptively assigned an effective hyperparameter configuration.

5 Evaluation

In this section, we evaluate the performance of AgileMFC using three real-world datasets. We aim to answer the following research questions (RQs):

- **RQ1:** How does AgileMFC perform on failure classification?
- **RQ2:** Does each component contribute to AgileMFC's performance?
- **RQ3:** What is the impact of different hyperparameters?

5.1 Settings

5.1.1 Datasets. To evaluate the performance of AgileMFC, we conduct extensive experiments on three datasets collected from three microservice systems under different business backgrounds and architectures. We split the datasets into training and testing sets based on temporal precedence, where earlier data is used for training and later data serves as the test set. Detailed dataset information is listed in Table 2. The datasets used in this study are as follows:

Generic AIOps Atlas (GAIA)[3]: GAIA is collected from a simulation environment consisting of 10 microservices, two database services (MySQL and Redis), and five host machines. Operators injected five types of failures, including system failures (System stuck and Process crash) and service failures (Login failure, File missing, and Access denied). The dataset includes more than 56 million metrics, 87 million logs, and 2.3 million traces. GAIA is a high-quality benchmark dataset that supports a wide range of Operation and Maintenance (O&M) automation tasks.

TrainTicket [28] (TT)[4]: The TT dataset is collected from a ticketing system with 41 microservices, of which 27 engage in frequent interactions. It supports operations including ticket booking, checking, and payment processing. With three types of injected failures and over 127 million metrics, 6.7 million logs, and 48,000 traces, TT reflects the complex interactions and high transaction volumes typical of real-world service systems.

SocialNetwork [4] (SN): The SN dataset simulates a social networking platform with 21 microservices, with 12 dedicated to core business. SN contains 2.4 million metrics, 1.5 million logs, and 126,000 traces, with three injected failure types. This dataset is used for analyzing user interactions, content creation, etc, making it ideal for studying microservice interaction and failure prediction.

Table 2: Detailed information of Datasets

	#Instances	#Failure types	Modality	#Samples
GAIA	10	5	Metric	56,684,196
			Log	87,974,577
			Trace	2,321,280
TT	27	3	Metric	127,171,863
			Log	6,768,927
			Trace	48,296
SN	12	3	Metric	2,439,072
			Log	1,468,692
			Trace	126,384

5.1.2 Metrics. We employ the weighted average *precision*, *recall* and *F1-score* to evaluate the performance of AgileMFC and baselines. The *precision*, *recall* and *F1-score* are calculated based on True Positives (TP), False Positives (FP), and False Negatives (FN). The calculation is given by F1-score = $\frac{2 \cdot precision \cdot recall}{precision + recall}$, where $precision = \frac{TP}{TP+FP}$, $recall = \frac{TP}{TP+FN}$.

The weighted average of these metrics is intended to ensure a comprehensive assessment of performance. Also, to minimize the effect of randomness, each experiment was repeated five times and the results were averaged.

5.1.3 Baselines. We use four baseline approaches for comparison, including two single-modal-based approaches and two multi-modal-based approaches.

Cloud19 [23] is a supervised framework designed to identify failure types using system logs.

LogCluster [11] employs unsupervised clustering techniques to identify failure types by grouping the log event based on similarity.

CloudRCA [26] is a root cause analysis approach that leverages cloud service metrics and logs to diagnose failure.

DiagFusion [24] integrates multi-modal data including logs, metrics, and traces through fasttext, and adopts a GNN for accurate failure diagnosis. DiagFusion presents the SoTA performance in existing works.

5.1.4 Implementation. AgileMFC is implemented in PyTorch and all of the experiments are conducted on a server with one Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz, and 32 GB RAM. For hyperparameter configuration, the number of experts is 8 for GAIA, 3 for TT, and 5 for SN, the number of Transformer blocks is 4 for GAIA, 1 for TT, and 2 for SN. We apply the Adam [7] optimizer and ReduceLROnPlateau to adjust the learning rate dynamically. The

[3]https://github.com/CloudWise-OpenSource/GAIA-DataSet
[4]Both TT and SN were collected by Eadro [8]

Table 3: The average precision, recall, and F1-score of different approaches on three datasets.

Approach	GAIA			TT			SN		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Cloud19 [23]	0.774	0.774	0.756	0.230	0.353	0.277	0.396	0.375	0.375
LogCluster [11]	0.615	0.477	0.336	0.435	0.450	0.441	0.186	0.300	0.220
CloudRCA [26]	0.436	0.453	0.357	-	-	-	0.692	0.462	0.478
DiagFusion [24]	0.806	0.829	0.839	0.710	0.700	0.696	0.455	0.300	0.325
AgileMFC	**0.958**	**0.924**	**0.939**	**0.751**	**0.720**	**0.728**	**0.872**	**0.620**	**0.701**

'-' denotes "out of memory" error.

initial learning rate is set to 0.001, with a batch size of 16 and 10 epochs for training.

5.2 *RQ1:* Performance of AgileMFC

Table 3 lists the average precision, recall, and F1-score of AgileMFC and four baseline approaches on the three datasets. AgileMFC consistently outperforms all baselines on all datasets, achieving the highest average F1-score of 0.939, 0.728, and 0.701, respectively.

On GAIA, AgileMFC shows exceptional performance with the average precision, recall, and F1-score all-surpassing 0.92, demonstrating its robustness in handling large-scale data with diverse failure types. On TT and SN, AgileMFC exceeds the baseline approaches by at least 0.2 in terms of the average F1-score. The superior performance of AgileMFC can be attributed to the efforts of the modality-aware representation network, where the Transformer block-based expert network can focus on the failure-sensitive events through the self-attention mechanism, and the gating mechanism flexibly merges the outputs of the experts according to the different modalities. The AgileMFC's performance on TT and SN trails GAIA slightly, which is understandable given that the sample sizes of TT and SN are much smaller than GAIA.

It is also noteworthy that multi-modal-based approaches generally outperform single-modal-based baselines, as they integrate diverse sources of data, leading to better classification performance. However, in certain cases, such as Cloud19 outperforming CloudRCA on GAIA, single-modal approaches can be better, especially when one modality carries more weight in the system's failure. This reinforces the importance of flexible trade-offs between the characteristics of different modalities in microservice failure classification.

In brief, AgileMFC demonstrates efficient performance in the failure classification task across all datasets, improving the average F1-score (across the three datasets) by 12.4%, 9.1%, and 11.8% compared to the 2nd-best baseline. These improvements highlight AgileMFC's value as a reliable tool for supporting downstream tasks in microservice system O%M, such as root cause localization and troubleshooting.

5.3 *RQ2:* Ablation Study

This part conducts a comprehensive ablation study to evaluate the contribution of each component of AgileMFC. We create seven variants of AgileMFC and perform a series of controlled experiments. The variants are described as follows:

AgileMFC using CNN: This variant replaces the Transformer block with a CNN to investigate the effect of multi-head attention on overall performance.

AgileMFC w/o gating: In this variant, we remove the gating mechanism to examine its contribution to AgileMFC's performance.

AgileMFC w/o sharing-gate: This variant removes the sharing-gate while retaining the modality-gate, allowing us to assess the role of cross-modal feature sharing in the gating mechanism.

AgileMFC w/o modality-gate: By eliminating the modality-gate while retaining the sharing-gate, we can better understand how modality features influence the gating process and performance.

AgileMFC w/o TF-IDF weighting: To study the importance of event weighting, we remove the TF-IDF weighting module, which is used to highlight important events in the model.

AgileMFC using KNN: To observe how the dense representations affect classification performance, we replace the fully connected layer with a KNN model.

AgileMFC using Random Forest: This setup of variant replaces the fully connected layer with a Random Forest model to investigate whether a tree-based ensemble approach could perform better in the decision process.

Table 4 summarizes the results, demonstrating that AgileMFC achieves higher performance than all variants across the datasets. Clearly, each component contributes to the performance of AgileMFC. Performance declines when the Transformer block is replaced, as self-attention mechanisms are particularly effective at capturing failure-sensitive representations, whereas CNNs struggle to extract such complex and nuanced features. When the gating mechanism is removed, including the sharing-gate or the modality-gate individually, the performance declines, highlighting the crucial role of the gating mechanism in dynamically assigning expert outputs. Specifically, removing the sharing-gate hinders the model's ability to share cross-modal information, while eliminating the modality-gate disrupts the correct allocation of weights to different modalities. Furthermore, excluding the TF-IDF weighting module impaired performance, as it limited the model's capacity to merge rich semantic embeddings, thereby weakening its ability to understand context. Finally, replacing the fully connected layer with KNN and Random Forest results in lower performance, as both approaches lack the ability to learn hierarchical representations and optimize parameters through backpropagation.

Table 4: The average precision, recall, and F1-score of AgileMFC and different model variants

Approach	GAIA			TT			SN		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
use CNN	0.928	0.894	0.908	0.662	0.610	0.624	0.695	0.500	0.538
w/o gating	0.941	0.913	0.926	0.590	0.570	0.568	0.695	0.500	0.536
w/o sharing-gate	0.930	0.896	0.910	0.687	0.660	0.668	0.703	0.500	0.546
w/o modality-gate	0.937	0.903	0.917	0.680	0.650	0.656	0.710	0.560	0.590
w/o TF-IDF weighting	0.886	0.833	0.857	0.710	0.640	0.657	0.710	0.580	0.588
use KNN	0.815	0.845	0.829	0.590	0.400	0.372	0.471	0.400	0.368
use Random Forest	0.844	0.882	0.857	0.451	0.450	0.450	0.300	0.300	0.300
AgileMFC	**0.958**	**0.924**	**0.939**	**0.751**	**0.720**	**0.728**	**0.872**	**0.620**	**0.701**

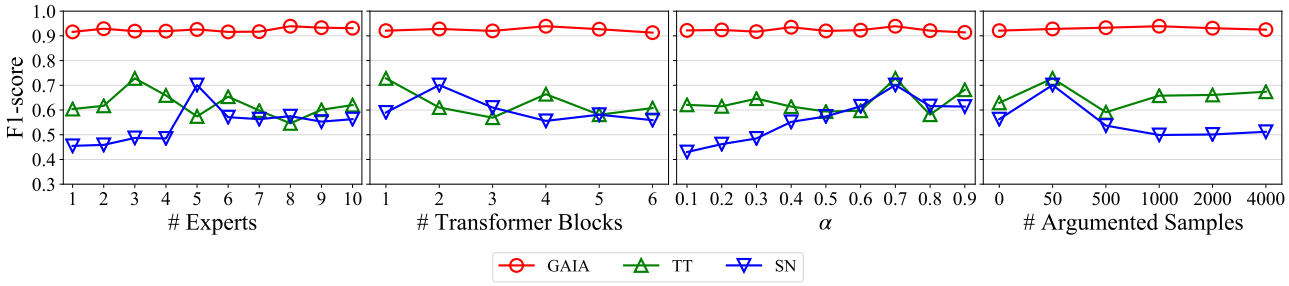


Figure 3: The performance of AgileMFC under different hyperparameters.

5.4 *RQ3:* Hyperparameters Sensitivity

We discuss the effect of four hyperparameters of AgileMFC. Fig. 3 shows how the average best F1-score changes with different hyperparameters.

5.4.1 The Number of Experts. The number of experts affects the feature learning ability of the MMoE network. Note that the MMoE is degraded to a shared-bottom model with only one expert. As the number of experts increases, MMoE has a greater capacity to capture the complex features of the modality, while more experts sacrifice the simplicity of the model, spending extra time in training. Based on our experiments, we recommend setting the number of experts between 3 and 8 to strike a good balance between representation performance and computational efficiency.

5.4.2 The Number of Transformer Blocks. As the block number of Transformer varies from 1 to 6, the performance of AgileMFC on GAIA is relatively stable with a slight increase, peaking at the fourth block. In contrast, the performance of TT and SN fluctuates more prominently, with a notable decline observed after reaching their respective optimal block numbers (1 for TT and 2 for SN). These observations suggest that while adding more blocks can enhance feature learning in certain cases, excessive complexity may lead to overfitting or reduced generalization.

5.4.3 The Coefficient of Sharing Gate. In Equ. 5, MMoE regulates the participation of sharing-gate via a coefficient α. We conducted experiments by varying α from 0.1 to 0.9 to observe its effect on performance. As depicted in Fig. 3, AgileMFC attains its optimal performance when α is equal to 0.7. Deviations from this value, either higher (closer to 1) or lower (closer to 0) of α, lead to suboptimal performance, consistent with the findings in § 5.3.

5.4.4 The Number of Augmented Samples. In AgileMFC, we augment the sample size of the dataset based on textual semantics, and experiments indicate that this improves AgileMFC's performance. However, textual augmentation must be subtle to capture semantic coherence and linguistic nuances compared to image data augmentation. The results on TT and SN demonstrate that inappropriate augmentation can lead to nonsensical or misleading data, thereby potentially impairing model performance.

6 Discussion

Limitations. AgileMFC requires labeled failure data, i.e., typical types observed in large-scale microservice systems, to classify failures. Attaining such annotation could be expensive and may require long-term operation recording. This challenge might be relieved by exploring semi-supervised learning techniques, which are capable of reducing the reliance on large-scale labeled data. On the other hand, although typical failures can cover a vast majority of real-world scenarios, there are still unknown and unseen failure types during the development of new microservices. We note that this is a common drawback of supervised learning, and one could refer to transfer learning for mitigating such cold start pitfall in diagnosing new systems and unseen issues.

Implications. The MoE structure is evaluated in this work to be effective, showing the SoTA performance for failure classification, in analyzing multi-modal monitoring data from multiple sources. The underlying factors are dealing with heterogeneous data with

more than one expert, thus implicitly matching tunable experts with different task cases. We believe such attempts could shed light on anomaly detection and root cause identification in AIOps of large systems by assembling the wisdom of a bunch of experts. Since high-volume microservices are expected following the increasing deployment of microservices, future work may focus on incorporating more data modalities and dynamic model updating strategies to accommodate the dynamic nature of microservice interactions.

7 Conclusion

Automatic failure classification is critical to maintaining the reliability of microservice systems. In this work, we propose AgileMFC, a novel framework that flexibly combines the deep features of multi-modal monitoring data for microservice failure classification. In design, it novelly adopts a modality-aware representation network that both independently extracts differential expertise and jointly combines them towards event-wise features. The split and merge design allows this module to retain the failure sensitivity of events with different modalities. It also provides a two-stage training mechanism for learning the representation and classification independently. Experiments on three datasets demonstrate the effectiveness of different components of AgileMFC and show that it outperforms four baselines in all the test cases.

Acknowledgments

This work is supported by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3027 and 2023RC3139, the Graduate Research Innovation Project of Hunan Province under Grant CX20230912, the National Natural Science Foundation of China under Grant 62272062, and the Natural Science Foundation of Hunan Province under Grant 2025JJ50373.

References

[1] Yang Cai, Biao Han, Jinshu Su, and Xiaoyan Wang. 2021. Tracemodel: An automatic anomaly detection and root cause localization framework for microservice systems. In *Proc. of 17th International Conference on Mobility, Sensing and Networking (MSN)*. 512–519.

[2] Yuhua Chen, Dongqi Xu, Ningjiang Chen, and Xu Wu. 2023. FRL-MFPG: Propagation-aware fault root cause location for microservice intelligent operation and maintenance. *Information and Software Technology* 153 (2023), 107083.

[3] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proc. of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[4] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proc. of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. 3–18.

[5] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1387–1397.

[6] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *Proc. of IEEE international conference on web services (ICWS)*. 33–40.

[7] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[8] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *Proc. of IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1750–1762.

[9] Alexander Lercher, Johann Glock, Christian Macho, and Martin Pinzger. 2024. Microservice API Evolution in Practice: A Study on Strategies and Challenges. *Journal of Systems and Software* 215 (2024), 112110.

[10] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. 2022. Swisslog: Robust anomaly detection and localization for interleaved unstructured logs. *IEEE Transactions on Dependable and Secure Computing (TDSC)* 20, 4 (2022), 2762–2780.

[11] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. 2016. Log clustering based problem identification for online service systems. In *Proc. of the 38th International Conference on Software Engineering Companion (ICSE-Companion)*. 102–111.

[12] Dewei Liu, Chuan He, Xin Peng, Fan Lin, Chenxi Zhang, Shengfang Gong, Ziang Li, Jiayu Ou, and Zheshun Wu. 2021. Microhecl: High-efficient root cause localization in large-scale microservice systems. In *Proc. of IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 338–347.

[13] Fengrui Liu, Yang Wang, Zhenyu Li, Rui Ren, Hongtao Guan, Xian Yu, Xiaofan Chen, and Gaogang Xie. 2022. MicroCBR: Case-Based Reasoning on Spatio-temporal Fault Knowledge Graph for Microservices Troubleshooting. In *International Conference on Case-Based Reasoning (ICCBR)*. 224–239.

[14] Dominik Scheinert, Alexander Acker, Lauritz Thamsen, Morgan K Geldenhuys, and Odej Kao. 2021. Learning dependencies in distributed cloud applications to identify and localize anomalies. In *Proc. of 2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*. 7–12.

[15] Bikash Sharma, Praveen Jayachandran, Akshat Verma, and Chita R Das. 2013. CloudPD: Problem determination and diagnosis in shared dynamic clouds. In *Proc. of 2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 1–12.

[16] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. LogKG: Log Failure Diagnosis through Knowledge Graph. *IEEE Transactions on Services Computing (TSC)* (2023).

[17] Yongqian Sun, Binpeng Shi, Mingyu Mao, Minghua Ma, Sibo Xia, Shenglin Zhang, and Dan Pei. 2024. ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1183–1194.

[18] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[19] Ruibo Wang, Kai Lu, Juan Chen, Wenzhe Zhang, Jinwen Li, Yuan Yuan, Pingjing Lu, Libo Huang, Shengguo Li, and Xiaokang Fan. 2021. Brief introduction of TianHe exascale prototype system. *Tsinghua Science and Technology* 26, 3 (2021), 361–369.

[20] Canhua Wu, Nengwen Zhao, Lixin Wang, Xiaoqin Yang, Shining Li, Ming Zhang, Xing Jin, Xidao Wen, Xiaohui Nie, Wenchi Zhang, et al. 2021. Identifying root-cause metrics for incident diagnosis in online service systems. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. 91–102.

[21] Yuxia Xie, Kai Yang, and Pan Luo. 2021. Logm: Log analysis for multiple components of hadoop platform. *IEEE Access* 9 (2021), 73522–73532.

[22] Dong Young Yoon, Ning Niu, and Barzan Mozafari. 2016. Dbsherlock: A performance diagnostic tool for transactional databases. In *Proc. of the 2016 international conference on management of data*. 1599–1614.

[23] Yue Yuan, Wenchang Shi, Bin Liang, and Bo Qin. 2019. An approach to cloud execution failure diagnosis based on exception logs in openstack. In *Proc. of the 12th International Conference on Cloud Computing (CLOUD)*. 124–131.

[24] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibo Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, et al. 2023. Robust failure diagnosis of microservice system through multimodal data. *IEEE Transactions on Services Computing (TSC)* 16, 6 (2023), 3851–3864.

[25] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *arXiv preprint arXiv:2407.01710* (2024).

[26] Yingying Zhang, Zhengxiong Guan, Huajie Qian, Leili Xu, Hengbo Liu, Qingsong Wen, Liang Sun, Junwei Jiang, Lunting Fan, and Min Ke. 2021. CloudRCA: A root cause analysis framework for cloud computing platforms. In *Proc. of the 30th ACM International Conference on Information & Knowledge Management (CIKM)*. 4373–4382.

[27] Na Zhao, Biao Han, Ruidong Li, Jinshu Su, and Cong Zhou. 2023. A Multivariate KPIs Anomaly Detection Framework With Dynamic Balancing Loss Training. *IEEE Transactions on Network and Service Management* 20, 2 (2023), 1418–1429.

[28] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proc. of the 40th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 323–324.